

Battle Scanner — Consumer App Improvement Plan

Scope: Everything except model/AI quality. Focus is on UX, reliability, performance, and polish. All file references are relative to `consumer/src/`.

Priority 1 — Bug Fixes (Do These First)

1.1 Add request timeout to `/api/detect`

File: `pages/ScanPage.tsx`

Problem: If the backend hangs (YOLO subprocess crash, GPU OOM, etc.), the user sees the scan animation forever with no way out except killing the app.

Fix: Use `AbortController` with a ~30s timeout.

```
tsx

const controller = new AbortController();
const timeout = setTimeout(() => controller.abort(), 30000);

try {
  const response = await axios.post('/api/detect', formData, {
    signal: controller.signal,
    headers: { 'Content-Type': 'multipart/form-data' }
  });
} catch (err) {
  if (axios.isCancel(err)) {
    setError('Scan timed out — please try again');
  } else {
    // existing error handling
  }
} finally {
  clearTimeout(timeout);
}
```

1.2 Fix stale closure in CameraCapture stream cleanup

File: `components/CameraCapture.tsx:44`

Problem: The cleanup function in `useEffect` captures `stream` when it's still `null`, so switching camera modes can leak the old stream.

Fix: Use a `ref` to hold the stream so cleanup always has the live value.

tsx

```
const streamRef = useRef<MediaStream | null>(null);

const startCamera = useCallback(async () => {
  if (streamRef.current) {
    streamRef.current.getTracks().forEach(t => t.stop());
  }
  const newStream = await navigator.mediaDevices.getUserMedia({ ... });
  streamRef.current = newStream;
  setStream(newStream); // keep state for UI if needed
}, [facingMode]);

useEffect(() => {
  startCamera();
  return () => {
    streamRef.current?.getTracks().forEach(t => t.stop());
  };
}, [facingMode]);
```

1.3 Verify/fix animate-scan-line class

File: `components/CameraCapture.tsx:96`

Problem: `animate-scan-line` is used in JSX but may not be defined in Tailwind config, meaning the scanning line animation silently does nothing.

Fix: Either confirm it's in `tailwind.config.js` under `extend.animation`, or replace with a standard inline `@keyframes` in the component (as `ScanAnimation.tsx` already does correctly).

1.4 Normalise faction display names

File: `components/ArmySummary.tsx`, `components/ResultsCard.tsx`

Problem: Faction names are displayed raw from the backend string. If the backend sends `"space_marines"` or `"adeptus_mechanicus"`, that's what shows in the UI.

Fix: Add a utility function and apply it at display time.

tsx

```
// utils/factionDisplay.ts
export function formatFactionName(raw: string): string {
  return raw
    .replace(/_/g, ' ')
    .replace(/\b\w/g, c => c.toUpperCase());
}

// "space_marines" → "Space Marines"
// "adeptus_mechanicus" → "Adeptus Mechanicus"
// "Space Marines" → "Space Marines" (already clean — idempotent)
```

Apply in both components: `formatFactionName(faction.faction)` instead of `faction.faction` directly.

Priority 2 — UX Improvements

2.1 Retry button after failed scan

File: `pages/ScanPage.tsx`

Problem: On error, the user sees an error message but then has to manually re-select or re-take their photo.

Fix: Add a "Try Again" button that re-submits the same image without requiring them to re-capture.

```
tsx

const [pendingFile, setPendingFile] = useState<File | null>(null);

// Store file on selection, re-use on retry
const handleRetry = () => {
  if (pendingFile) handleSubmit(pendingFile);
};
```

2.2 Show inference time on results page

File: `pages/ResultsPage.tsx`

Problem: `inferenceTimeMs` is already in `DetectionResult` but not displayed anywhere. Users notice this kind of detail — it builds trust and feels like a real tool.

Fix: Add a small badge in the results header.

```
tsx
```

```
<span className="text-xs text-gray-400">
  Scanned in {(results.inferenceTimeMs / 1000).toFixed(1)}s
</span>
```

2.3 Confidence threshold filter in results

File: `pages/ResultsPage.tsx`

Problem: Low-confidence detections (e.g. 15%) are shown on equal footing with 90%+ detections, which makes the results noisy and untrustworthy.

Fix: Add a confidence slider or toggle to filter the bbox overlay and faction cards.

```
tsx

const [minConfidence, setMinConfidence] = useState(0.4);

const filteredDetections = results.detections.filter(
  d => d.confidence >= minConfidence
);
```

Default to 40%. Show a small "Showing X of Y detections" counter. This also reduces visual clutter on the canvas overlay.

2.4 Empty state when no miniatures detected

File: `pages/ResultsPage.tsx`

Problem: When `totalDetected === 0`, the "No miniatures detected" text renders below the (empty) canvas and faction area, which looks broken rather than intentional.

Fix: Short-circuit to a proper full-screen empty state.

```
tsx

if (results.totalDetected === 0) {
  return (
    <div className="flex flex-col items-center justify-center h-full gap-4">
      <img src={scannedImage} className="w-full max-w-sm rounded-lg opacity-50" />
      <p className="text-gray-400 text-center">No miniatures detected.<br/>Try a clearer photo with better lighting.</p>
      <button onClick={onScanAnother}>Try Again</button>
    </div>
  );
}
```

2.5 Loading state for camera initialisation

File: `components/CameraCapture.tsx`

Problem: On first load, there's a brief moment before the camera stream appears — the user sees a blank black box with no feedback.

Fix: Track a `cameraLoading` state and show a spinner/message until the stream is live.

```
tsx

const [cameraReady, setCameraReady] = useState(false);

videoRef.current.onloadedmetadata = () => setCameraReady(true);

// In JSX:
{!cameraReady && (
  <div className="absolute inset-0 flex items-center justify-center">
    <p className="text-gray-400 text-sm">Starting camera...</p>
  </div>
)}
```

2.6 Handle camera permission denial gracefully

File: `components/CameraCapture.tsx`

Problem: If the user denies camera permission, a raw browser error propagates with no user-friendly message.

Fix: Catch `NotAllowedError` and `NotFoundError` specifically.

```
tsx

} catch (err: any) {
  if (err.name === 'NotAllowedError') {
    setError('Camera permission denied. Please allow camera access in your browser settings.');
```

```
  } else if (err.name === 'NotFoundError') {
    setError('No camera found on this device.');
```

```
  } else {
    setError('Could not start camera.');
```

```
  }
}
```

2.7 Photo upload drag-and-drop improvement

File: `components/PhotoUpload.tsx`

Problem: The `fetch(preview).then(r => r.blob())` roundabout approach to recover the Blob loses the original `File` object. The original `File` should just be kept in state.

Fix: Store the `File` directly.

```
tsx

const [file, setFile] = useState<File | null>(null);

const handleFileChange = (f: File) => {
  setFile(f);
  const reader = new FileReader();
  reader.onload = e => setPreview(e.target?.result as string);
  reader.readAsDataURL(f);
};

// On submit: use file directly — no fetch(preview) roundtrip
const formData = new FormData();
formData.append('image', file);
```

Priority 3 — Polish & Visual Quality

3.1 Faction display icons/badges

Context: The app has `factionThemes.ts` in the frontend annotator. The consumer app only uses `faction.color` for badge backgrounds.

Improvement: Add a small icon or symbol per faction to the ArmySummary badges and ResultsCard headers. Even a simple SVG faction symbol makes the results page feel far more finished to the target audience (Warhammer players).

3.2 Results canvas bbox label cleanup

File: `pages/ResultsPage.tsx`

Problem: Labels currently show the raw faction name. They should show the formatted name plus confidence percentage, and the label positioning should avoid being cut off at image edges.

Fix:

```
tsx
```

```
// Clamp label position to stay within canvas
const labelX = Math.min(bbox.x, canvas.width - 120);
const labelY = bbox.y > 20 ? bbox.y - 5 : bbox.y + bbox.height + 15;

ctx.fillText(
  `${formatFactionName(det.faction)} ${Math.round(det.confidence * 100)}%`,
  labelX,
  labelY
);
```

3.3 Scan animation polish

File: `components/ScanAnimation.tsx`

Problem: Cycling phrases are good, but the animation has no visual connection to Warhammer 40K aesthetics beyond the copy.

Improvement: Add the aquila icon from the Header, or a simple animated targeting reticle SVG. The aesthetic gap between "BATTLE SCANNER" branding and a generic spinner is noticeable.

3.4 Share/export results

Enhancement: Add a "Share Army" button on the results page that uses the [Web Share API](#) to share the canvas image (with bboxes drawn) plus a text summary.

```
tsx

const handleShare = async () => {
  const canvas = canvasRef.current;
  canvas.toBlob(async (blob) => {
    const file = new File([blob], 'army-scan.png', { type: 'image/png' });
    const summary = results.summary
      .map(s => `${formatFactionName(s.faction)}: ${s.count}`)
      .join(', ');

    await navigator.share({
      title: 'My Warhammer Army Scan',
      text: summary,
      files: [file]
    });
  });
};
```

This is one of the highest-impact features for virality — players will want to share results with their group.

3.5 PWA install prompt

File: `consumer/` (probably `App.tsx`) or a new component)

Context: The consumer app is already a PWA (`vite-plugin-pwa`). But most users won't know to install it.

Fix: Listen for `beforeinstallprompt` and surface a subtle "Add to Home Screen" banner after a successful scan (not on first visit).

```
tsx

useEffect(() => {
  window.addEventListener('beforeinstallprompt', (e) => {
    e.preventDefault();
    setInstallPrompt(e); // store for later
  });
}, []);

// Show banner after first successful scan
```

Priority 4 — Reliability & Edge Cases

4.1 Very large image handling

Problem: A user photographing a large army might take a 12MP photo. Sending that to the backend is slow, and YOLO inference on it may be very slow too.

Fix: Downscale before upload using a canvas in the browser. 1280px wide is plenty for detection.

```
tsx
```



```
const resizeImage = (file: File, maxWidth = 1280): Promise<Blob> => {
  return new Promise(resolve => {
    const img = new Image();
    img.onload = () => {
      const scale = Math.min(1, maxWidth / img.width);
      const canvas = document.createElement('canvas');
      canvas.width = img.width * scale;
      canvas.height = img.height * scale;
      canvas.getContext('2d').drawImage(img, 0, 0, canvas.width, canvas.height);
      canvas.toBlob(resolve, 'image/jpeg', 0.85);
    };
    img.src = URL.createObjectURL(file);
  });
};
```

4.2 Offline handling

Problem: The app is a PWA but there's no handling for when the user tries to scan while offline — the fetch just fails with a network error.

Fix: Check `navigator.onLine` before submitting and show a friendly message.

```
tsx

if (!navigator.onLine) {
  setError('No internet connection. Scanning requires a connection to the server.');
```

4.3 Multiple detections of same miniature

Problem: YOLO may produce overlapping detections for the same model at different confidence levels. The backend should handle NMS, but it's worth confirming the consumer UI doesn't show the same miniature counted twice in the faction summary.

Note: Verify `detectAndSummarize` in `yoloInferenceService.ts` applies proper NMS before building the summary. If it doesn't, duplicate detections inflate counts noticeably.

Summary Prioritisation

#	Item	Impact	Effort
1.1	Request timeout	High — prevents infinite spinner	Low
1.2	Camera stream leak fix	Medium — edge case bug	Low
1.3	Scan line animation fix	Low — cosmetic	Very Low
1.4	Faction name formatting	High — visible to every user	Very Low
2.1	Retry after error	High — core UX	Low
2.2	Inference time display	Low — nice detail	Very Low
2.3	Confidence threshold filter	High — improves result clarity	Medium
2.4	Empty state redesign	Medium — polish	Low
2.5	Camera init loading state	Medium — first impression	Low
2.6	Permission denial handling	High — breaks app if hit	Low
2.7	File object cleanup	Low — no user-visible effect	Very Low
3.1	Faction icons	High — audience expects this	Medium
3.2	Label positioning	Medium — polish	Low
3.3	Scan animation visual	Low — cosmetic	Medium
3.4	Share results	Very High — viral feature	Medium
3.5	PWA install prompt	Medium — retention	Low
4.1	Image resize before upload	High — real-world perf	Low
4.2	Offline handling	Medium — PWA expectation	Very Low
4.3	NMS verification	High — data integrity	Low (investigation)

Recommended Order of Attack

1. **Fixes first** (1.1, 1.2, 1.3, 1.4) — small, safe, no regressions
2. **Core UX** (2.1, 2.3, 2.4, 2.6) — biggest user-visible quality improvements
3. **Image resize** (4.1) — real-world necessity before sharing with anyone
4. **Share feature** (3.4) — highest impact for getting the app actually used
5. **Polish** (3.1, 3.5, 2.5) — once the fundamentals are solid